Práctica integrada ISO

ESIBuy: Plataforma de Comercio Electrónico.

Gestión de Proyectos Software+ Seguridad de Sistemas Software + Procesos de Ingeniería del Software + Calidad de Sistemas Software

Se desea crear un sistema web de **comercio electrónico** que permita a los usuarios registrados buscar, consultar, comprar y valorar productos en un entorno seguro, escalable y accesible desde cualquier navegador web. El sistema debe permitir las opciones básicas de una plataforma de este tipo: administración de productos, gestión de pedidos, gestión del carrito de compra, configuración de productos favoritos, valoraciones de productos y seguimiento de envíos.

La práctica debe crearse utilizando **Java y Spring Framework** para el backend. El desarrollo deberá aplicar buenas prácticas de ingeniería del software, entre otras:

- Gestión del proyecto usando SCRUM.
- Planificación de tareas usando paneles y la herramienta Azure DevOps.
- Gestión de la configuración y control de versiones con Git.
- Uso de GitHub Copilot como apoyo al desarrollo dirigido con IA, principalmente en la parte frontend.
- Definición previa de especificaciones, prompts, instrucciones o *skills* que guíen el uso de la IA durante el desarrollo.
- Utilización de patrones de ingeniería del software.
- Desarrollo de software utilizando Test Driven Development (TDD).
- Aplicación de la estrategia de pruebas mediante técnicas dirigidas por IA.
- Como gestor de BBDD se usará MongoDB.

Además, el uso de IA generativa deberá estar documentado. Para ello, el equipo deberá definir un conjunto de especificaciones o skills de desarrollo, donde se indique cómo se utilizará GitHub Copilot para apoyar tareas como:

- Generación de componentes de interfaz.
- Mejora de la usabilidad y del aspecto visual de la interfaz.
- Generación de pruebas unitarias o de integración.

El uso de Copilot no sustituirá la responsabilidad del equipo sobre el código generado. Todo código generado con apoyo de IA deberá ser revisado, entendido, adaptado y probado por los estudiantes.

Existen los siguientes roles:

- Administrador: encargado de gestionar usuarios, vendedores, categorías, incidencias y configuración general de la plataforma. Puede acceder en modo “Sólo lectura” a la información gestionada por los vendedores.
- Vendedor: encargado de dar de alta y gestionar productos, precios, stock, disponibilidad, ofertas y pedidos asociados a sus productos.
- Cliente: puede buscar productos, consultar detalles, añadir productos al carrito, realizar compras, gestionar direcciones de envío, valorar productos comprados y consultar el estado de sus pedidos. Existen dos tipos de cliente: Premium y Estándar.

La aplicación web tiene los siguientes requisitos:

- Para acceder a la herramienta, los usuarios deberán identificarse ante el sistema con un usuario y contraseña. Para eso debe utilizarse el patrón RBAC.
- Debe permitir a los usuarios registrarse en el sistema, iniciar sesión, gestionar su perfil, sus direcciones, sus métodos de pago y sus preferencias.
- Debe permitir a los clientes buscar productos, filtrarlos por categoría, precio, valoración o disponibilidad, consultar su información detallada y añadirlos al carrito de compra.
- Debe permitir a los clientes realizar pedidos, consultar su historial de compras, cancelar pedidos cuando sea posible y hacer seguimiento del estado del envío.
- Debe permitir a los clientes valorar productos únicamente cuando hayan realizado una compra previa.
- Debe permitir a los clientes guardar productos como favoritos o en listas de deseos.
- Debe permitir a los vendedores crear, modificar, consultar, activar/desactivar y eliminar productos.
- Debe permitir a los vendedores consultar estadísticas básicas de ventas, valoraciones y stock disponible.
- La aplicación debe tener un módulo de gestión de usuarios, solamente accesible por el administrador, con las funciones: crear otros administradores, modificar, consultar, bloquear/desbloquear y eliminar usuarios.
- Debe establecerse una política de contraseñas segura.
- Debe contar con un registro de usuario seguro y un sistema de recuperación de contraseñas.
- Debe contar con mecanismos de control y seguridad de sesión.
- La aplicación deberá contar con una conexión segura de tipo HTTPS.
- La información sensible deberá estar protegida de forma adecuada en la BBDD.
- Los datos relativos a pagos no deberán almacenarse en claro y deberán gestionarse siguiendo buenas prácticas de seguridad.
- Debe tener el mínimo de vulnerabilidades e identificarlas adecuadamente. El número de defectos, bugs y vulnerabilidades de severidad bloqueante y crítica debe ser 0.
- Se debe realizar un Análisis y Gestión de Riesgos (AGR) del sistema en cumplimiento con la norma ISO/IEC 27001:2022.
- Se debe cumplir con los principios de seguridad especificados en el documento CCN-CERT BP/28 y en las guías de buenas prácticas vistas en clase.
- Como medida de protección adicional, se deberá contar con un segundo factor de autenticación y un tercer factor de autenticación (3FA).
- Se deberán incorporar medidas de seguridad a nivel de BBDD para evitar el robo de información en caso de fallo de las capas superiores.
- Se deberá identificar vulnerabilidades de seguridad en el código fuente mediante utilidades IA, corrigiéndolas y entregando un informe final con los resultados.

En relación con las pruebas y la calidad del software:

- Se mantendrá un enfoque TDD, de forma que al menos el 50% de las funcionalidades implementadas deberán desarrollarse siguiendo Test Driven Development, utilizando JUnit para las pruebas del backend.
- Se deberá realizar la validación del sistema mediante pruebas dirigidas o asistidas por IA.

- La estrategia de pruebas deberá incluir pruebas unitarias.
- Se podrán utilizar herramientas de IA para proponer o revisar casos de prueba, debiendo justificarse y documentarse su uso.
- Todo caso de prueba generado con apoyo de IA deberá ser revisado y validado por los desarrolladores.
- Con el objetivo de asegurar el correcto funcionamiento de la aplicación, las pruebas deberán cubrir al menos el 60% del código total.
- No se permitirá la existencia de código en la aplicación que no sea utilizado.
- Se debe proporcionar una matriz de trazabilidad entre los Casos de Prueba y cada una de las Historias de Usuario implementadas.
- De cara a facilitar los cambios en el código, se debe evitar la repetición de fragmentos de código, por lo que el código duplicado deberá ser inferior al 10%.
- También para mejorar la analizabilidad del código, se debe evitar la creación de métodos con una complejidad ciclomática excesiva y, en caso de existir, deberá justificarse.
- Se debe cumplir con las mejores prácticas de programación para los lenguajes de Java, HTML y CSS, reduciendo el número de defectos en el código y evitando defectos de severidad grave o crítica.
- Los resultados de calidad alcanzados en el Sprint 1 no deberán empeorar en los siguientes Sprints.
- Para verificar el cumplimiento de estos criterios de calidad se utilizarán herramientas como Sonarqube y SonarLint.

La práctica de laboratorio será desarrollada a lo largo del cuatrimestre y se realizarán reuniones periódicas con los profesores.

Mantenimiento:

- Añadir un sistema de recomendaciones de productos de acuerdo con las preferencias, historial de compras y búsquedas del cliente.
- Incorporar un sistema de alertas de bajada de precio o reposición de stock.
- La aplicación deberá poder estar disponible para dispositivos móviles.